

ACTIVIDAD 6 (Parte 2)

Estructuras Repetitivas, Funciones, Cadenas y Librería propia

Objetivos

- Evaluar los conocimientos adquiridos sobre estructuras repetitivas (while, for, do-while), funciones y manipulación de cadenas.
 - Desarrollar habilidades prácticas en la implementación de algoritmos utilizando funciones y ciclos.
 - Fomentar el uso de buenas prácticas de programación y modularización de código.
 - Practicar la implementación de algoritmos fundamentales mediante problemas reales que involucren cadenas y funciones.
 - Implementar librería propia
-

Competencias a Desarrollar

- Programación modular avanzada.
 - Manejo de estructuras repetitivas y funciones.
 - Validación y tratamiento de datos de entrada.
 - Resolución de problemas mediante funciones y ciclos.
 - Manipulación de cadenas de texto.
 - Desarrollo de librería propia `#include "mi_libreria.h"`
-

Requisitos Previos

- Conocimientos básicos de programación.
 - Conocimiento de expresiones lógicas, tipos de datos y estructuras condicionales.
 - Conocimiento de funciones, ciclos y cadenas.
 - Entorno de desarrollo configurado con GCC o G++.
 - Editor de texto configurado (VSCode).
-

Instrucciones

1. Abre Visual Studio Code y configura tu entorno para compilar programas en C/C++ usando GCC o G++.
2. Completa todos los ejercicios propuestos en lenguaje C/C++.
3. Nombra cada archivo fuente con extensión .cpp según las convenciones establecidas en clase (por ejemplo, iniciales_ACT6_p2_#.cpp, PNY_ACT6_p2.cpp).
4. Para cada ejercicio:
 - Captura una imagen del código fuente.
 - Captura una imagen de la ejecución del programa mostrando la salida en consola.
5. Elabora un documento Word que incluya:
 - Una portada con el nombre de la actividad, tu nombre, matricula y grupo.
 - Las capturas de pantalla del código.
 - Las capturas de pantalla de las ejecuciones.
 - Comentarios o explicaciones relevantes.
6. Exporta el documento a formato PDF.
7. Nombra el archivo PDF como INICIALES_PE_ACT6.PDF.
8. Entrega en la plataforma CLASSROOM:
 - El archivo PDF.
 - Archivo fuente (.cpp) funciones de todos los ejercicios.

Ejercicios

Generación de Patrones con cadenas

Desarrolle una función que genere un patrón en forma de pirámide. Por ejemplo, si el usuario ingresa "ENSENADA", el programa debe mostrar:

Patrón 1:

None

ENSENADA

Patrón 2:

None

E
N
S
E
A
D
A

Patrón 3:

None

A
D
A
N
E
S
N
E

Patrón 4:

None

ENSENADA
ENSENAD
ENSENA
ENSEN
ENSE
ENS
EN
E

Patrón 5:

None

ADANESNE

ADANESN

ADANES

ADANE

ADAN

ADA

AD

A

Patrón 6:

None

ENSENADA

NSEANDA

SENADA

ENADA

NADA

ADA

DA

A

Patrón 7:

None

ADANESNE

DANESNE

ANESNE

NESNE

ESNE

SNE

NE

E

Implementación:

- Utilice ciclos anidados para generar el patrón.
- Asegúrese de que el programa funcione correctamente para cada patrón.
- Crea y utiliza librería propia `#include "mi_libreria.h"` llamar desde el programa principal. La librería deberá contener la función `my_gets` para leer cadenas

Menú Principal

Desarrolle una función que presente un menú interactivo con las siguientes opciones:

El programa debe permitir al usuario seleccionar una opción y llamar a la función correspondiente.

Implementación:

- Utilice un ciclo `do-while` para mantener el menú activo hasta que el usuario seleccione la opción de salir.
- Valide la entrada del usuario para evitar errores.

Formato del Reporte

El reporte debe incluir las siguientes secciones:

1. **Portada:** Nombre de la actividad, tu nombre, matrícula y grupo.
2. **Introducción:** Breve descripción del propósito de la actividad.
3. **Teoría:** Breve explicación del tema que se esta tratando en la actividad
4. **Código:** Incluye bloques de código para cada ejercicio, acompañados de comentarios explicativos.
5. **Resultados:** Capturas de pantalla o texto con los resultados obtenidos al ejecutar los programas.
6. **Conclusiones:** Reflexión sobre lo aprendido y posibles mejoras.

Consideraciones Importantes

- Usa nombres descriptivos para variables y funciones.
- Comenta tu código para facilitar su comprensión.
- Valida las entradas del usuario para evitar errores.
- Prueba tu programa con diferentes valores de entrada para asegurar su correcto funcionamiento.

- Mantén una estructura clara y organizada en tu reporte.
-